



Vedi Pocket PC

Offline Zero-Latency PC Control & Mirror Suite

Full System Architecture & Technical Reference

A comprehensive deep-dive into the protocols, binary streaming engine, Clean Architecture core, CustomTkinter desktop manager, and React Native mobile client.

FASTAPI & PYTHON 3.10+

REACT NATIVE EXPO

100% OFFLINE LAN

DXGI HIGH-FPS SCREEN STREAM

ZERO-TRUST PIN PAIRING

WINDOWS 10 / 11

Table of Contents

- | | |
|--|---|
| 1. Executive Summary & Overview | 8. Desktop Controller & Process Manager |
| 2. System Topology & Architecture | 9. Mobile Client Application Engine |
| 3. Network Ports & Protocol Matrix | 10. Repository & File Layout |
| 4. Authentication & Easy-Connect Flow | 11. Configuration (.env & config.json) |
| 5. DXGI Screen Stream Engine & Framing | 12. Build, Packaging & Distribution |
| 6. Input Event Dispatch & Wire Schemas | 13. Diagnostics & Troubleshooting |
| 7. Complete REST API Specification | 14. Test Suite & Quality Verification |

1. Executive Summary & Overview

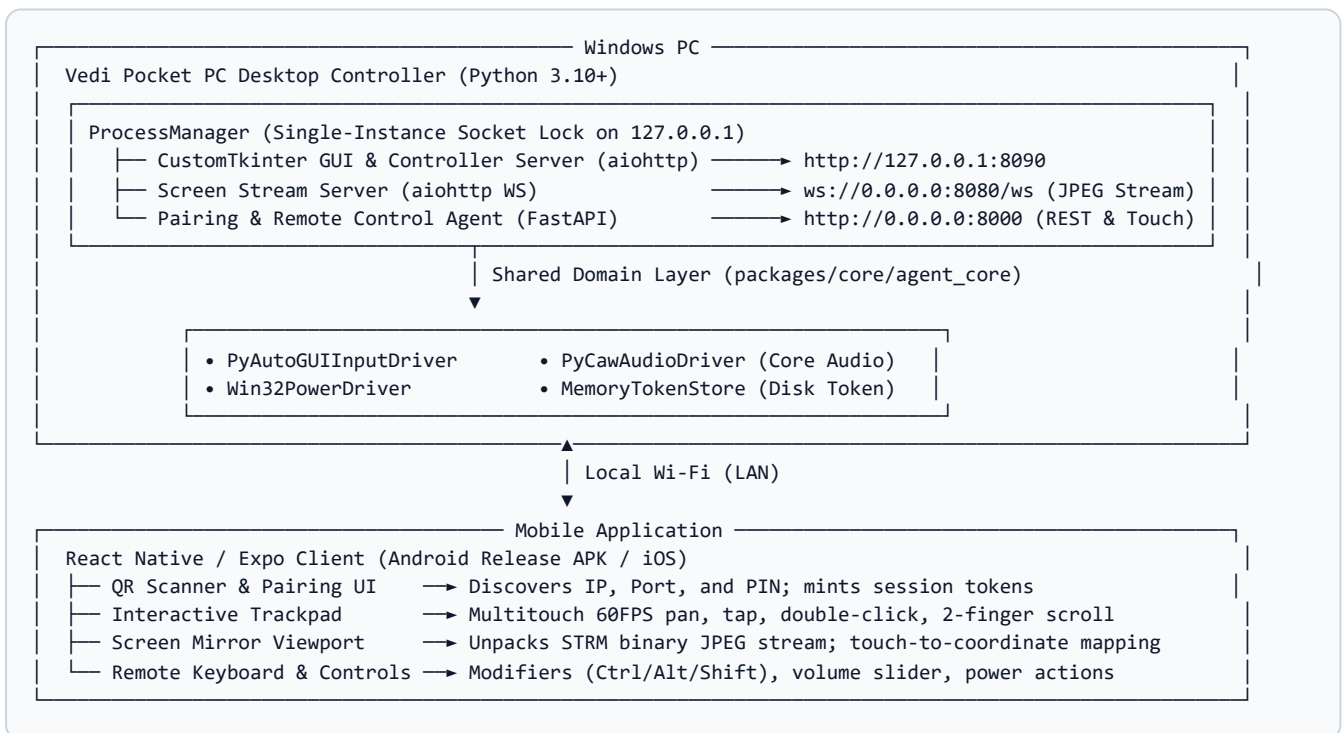
Vedi Pocket PC is an ultra-fast, offline PC remote control and display mirroring application suite designed for Windows 10 and 11. It transforms any Android or iOS smartphone into an interactive wireless trackpad, physical keyboard simulator, multimedia controller, and live screen mirror over local Wi-Fi.

Key Architectural Differentiators

- **Zero Cloud Relays:** 100% peer-to-peer over local network. No external server, internet access, or third-party telemetry.
- **Sub-50ms Input & Frame Latency:** Direct binary WebSocket streaming with DXGI desktop scraping and latest-wins frame queue dropping.
- **Zero-Trust Dynamic Pairing:** Secure 4-digit PIN generated per session, encoded into an instant camera QR code.
- **Clean Architecture Foundation:** Domain core decoupled into strict Entities, Ports, Adapters, and Use Cases inside `agent_core`.

2. System Topology & Architecture

The desktop suite operates as a single executable hosting three core subsystems via multi-threading and an embedded `aiohttp` / `FastAPI` runtime managed by `ProcessManager`:



3. Network Ports & Protocol Matrix

Port	Protocol	Component	Primary Responsibility
8090	HTTP / WS	Desktop Controller	CustomTkinter backend, management web dashboard, live log fan-out.
8080	WebSocket / HTTP	Screen Streamer	DXGI binary JPEG frame streaming, interactive viewport touch dispatch.
8000	HTTP REST / WS	FastAPI Agent	Zero-trust PIN pairing, system audio, power states, trackpad WebSocket.
8088	HTTP / WS	Metro Dev Server	Expo mobile bundle reloader (developer mode only).
5353	UDP (mDNS)	ZeroConf Advertiser	Broadcasts <code>_vedi-pocketpc._tcp.local.</code> for automatic LAN discovery.

4. Authentication & Easy-Connect Flow

The pairing mechanism enforces local network security without passwords or cloud logins:

1. **PIN Generation:** On startup, the control agent generates a random 4-digit PIN (`0000..9999`).
2. **QR Code Presentation:** The desktop UI renders a QR code encoding `<LAN_IP>:<PORT>:<PIN>` (e.g. `192.168.1.100:8000:4821`).
3. **Exchange (`POST /pair`):** The mobile client sends `{"pin": "4821"}` . If accepted, the agent mints an ephemeral session token plus a persistent `common_token` .
4. **Persistent Reconnect ("Easy-Connect"):** The agent persists `common_token` to `%LOCALAPPDATA%\PCRemoteAgent\common_token.txt` . The mobile app stores this token in `AsyncStorage` and reconnects automatically without re-entering the PIN.

5. DXGI Screen Stream Engine & Framing

Screen streaming runs over `ws://<IP>:8080/ws` using an optimized binary frame protocol.

Binary Framing Format

Magic Bytes (4 bytes) "STRM" (0x53 54 52 4D)	Payload Length (4 bytes) uint32 Big-Endian	Raw JPEG Image Bytes Variable Size (bytes)
---	---	---

Zero-Latency Queueing: The server writer uses a bounded queue of size 1 with a "latest-wins" discard policy. If the network interface cannot flush a frame in time, the stale frame is dropped immediately so the mobile screen never lags.

6. Input Event Dispatch & Wire Schemas

All user gestures received over WebSockets (ports 8000 and 8080) are parsed by `dispatch_table.py` and executed via `ControlInput` :

Event Type	Payload Parameters	Action Executed on Windows PC
mouse_move	dx, dy, sensitivity	Relative cursor move (PyAutoGUI <code>moveRel</code>)
mouse_move_to	x, y, duration	Absolute cursor placement (PyAutoGUI <code>moveTo</code>)
mouse_click	button, clicks, x?, y?	Left / right / middle click simulation
mouse_down / up	button, x?, y?	Physical mouse button hold / release (click-and-drag)
scroll	dy, dx?	Mouse wheel vertical and horizontal scroll
keyboard_type	text	Types a string sequence into active window
key_press	key (e.g. "enter" , "backspace")	Single key stroke press
hotkey	keys: ["ctrl", "alt", "del"]	Simultaneous key combo execution
ping	t?: number	Round-trip latency probe (replies with <code>pong</code>)

7. Complete REST API Specification

All authenticated endpoints require header: `Authorization: Bearer <token>` .

Pairing & System Endpoints

- `GET /health` (Public) → `{ "status": "ok", "agent_version": "1.0.0", "hostname": "VEDI-PC", "uptime_seconds": 3600 }`
- `POST /pair` (Public) → Body: `{ "pin": "1234" }` → `{ "status": "success", "token": "...", "common_token": "..." }`
- `GET /status` (Auth) → `{ "os": "Windows", "os_release": "11", "volume": 65, "battery": { "percent": 98, "plugged": true } }`
- `POST /system/lock` (Auth) → Locks Windows workstation.
- `POST /system/sleep` (Auth) → Puts Windows into low-power sleep.
- `POST /system/shutdown` (Auth) → Initiates timed Windows shutdown.

Audio & Media Control Endpoints (`/media/*`)

- `POST /media/volume` → Body: `{ "level": 75 }` → Direct Core Audio `IAudioEndpointVolume` adjustment.
- `POST /media/volume/up` & `/media/volume/down` → Step volume adjustment.
- `POST /media/volume/mute` → Toggle master mute.
- `POST /media/playpause` , `/next` , `/prev` → Global Windows media keys.
- `POST /media/type` → Body: `{ "text": "sample" }` → Types text into focused window.

8. Desktop Controller & Process Manager

Located in `apps/desktop/controller/` , the desktop controller is built with CustomTkinter and Python:

- **CustomTkinter GUI (`app.py`)**: Dark-themed native window displaying pairing QR codes, live IP/port stats, Metro server status, and real-time diagnostic log stream.
- **In-Process Thread Architecture**: Eliminates sub-processes by running the FastAPI agent and Screen Streamer on background daemon threads managed by `process_manager.py` .
- **Single-Instance Enforcement**: Binds a loopback socket on `127.0.0.1` to prevent duplicate instances.
- **Network Adapter Filter (`network.py`)**: Automatically filters out virtual adapters (Docker, WSL, VMware, Hyper-V) and prioritizes active physical Wi-Fi/Ethernet adapters.
- **Path Matrix (`paths.py`)**: Seamlessly handles development source trees, PyInstaller `onedir` bundles, and PyInstaller `onefile` extraction environments.

9. Mobile Client Application Engine

Located in `apps/mobile/app/` , the mobile client is built on React Native and Expo:

- **Multitouch Trackpad** (`trackpad.tsx`): Uses `react-native-gesture-handler` and `react-native-reanimated` for 60FPS fluid cursor motion, 1-finger left click, 2-finger right click, double-click, and 2-finger scroll.
- **Screen Viewport** (`DesktopViewport.tsx`): Unpacks binary `STRM` WebSocket frames into memory and renders with `expo-image` . Supports touch coordinate translation for direct desktop screen interaction, live latency meter, and screenshot capture.
- **Remote Keyboard** (`keyboard.tsx`): Full on-screen keyboard with modifier toggles (`Ctrl` , `Alt` , `Shift` , `Win`) and common hotkey macros.
- **Persistent State** (`deviceStore.ts`): Zustand store backed by `AsyncStorage` for cached connection tokens and paired device history.

10. Repository & File Layout

```
Vedi-Pocket-PC/
├── .env                                # Local development environment configuration
├── requirements.txt                    # Master Python dependencies list
├── apps/
│   ├── desktop/controller/           # CustomTkinter Desktop GUI & Process Manager
│   │   ├── app.py                    # Primary execution entry point
│   │   ├── process_manager.py        # Thread-based in-process runner & socket lock
│   │   ├── config.py / paths.py      # Config loader & multi-tier path resolver
│   │   ├── network.py / logging_setup.py # NIC discovery & rotating file logs
│   │   └── installer.iss / *.spec     # Inno Setup and PyInstaller build specs
│   ├── agent/server/                 # FastAPI Remote Control & Pairing Agent
│   │   └── main.py                   # FastAPI composition root & ZeroConf advertiser
│   ├── streamer/server/              # High-Speed Screen Streamer Engine
│   │   ├── main.py                   # aiohttp streaming composition root
│   │   ├── domain/capture.py         # DXGI frame capture via mss & PIL
│   │   └── presentation/ws_router.py # Binary frame broadcaster & touch handler
│   └── mobile/app/                   # React Native / Expo Mobile Client
│       ├── app/(tabs)/               # trackpad.tsx, screen.tsx, keyboard.tsx, controls.tsx
│       ├── components/DesktopViewport.tsx # Binary frame unpacker & touch coordinate mapper
│       └── src/ws/                   # client.ts (WS client) & pairing.ts (API client)
├── packages/
│   ├── core/agent_core/              # Shared domain logic, entities, ports, adapters
│   └── protocol/                     # Shared HTTP routers & WS dispatch table
├── infrastructure/                   # Centralized logging & mDNS network tools
├── scripts/                          # setup.bat, start.bat, build.bat, Uninstall.bat
└── tests/                           # Unit test suite for pairing, input, and controllers
```

11. Configuration (.env & config.json)

Variable	Default	Description
STREAM_HOST	0.0.0.0	Bind interface for screen streamer WebSocket.
STREAM_PORT	8080	TCP port for high-speed screen streaming.
STREAM_FPS	30	Target screen capture frame rate (10–60 FPS).
STREAM_JPEG_QUALITY	50	JPEG image compression factor (10–100).
STREAM_MAX_WIDTH	1280	Maximum streamed frame width in pixels.
STREAM_MAX_HEIGHT	720	Maximum streamed frame height in pixels.
BACKEND_PORT	8000	FastAPI pairing and control agent REST port.
CONTROLLER_PORT	8090	Desktop management HTTP server port.
EXPO_ENABLED	1	1 = Launch Metro dev server, 0 = Standalone mode.

12. Build, Packaging & Distribution

Master Build Commands

- **Developer Setup:** Run `.\scripts\setup.bat` to install Python wheels and npm dependencies.
- **Run from Source:** Run `.\scripts\start.bat` to boot the CustomTkinter GUI and all services.
- **Build Windows Standalone EXE:** Run `.\scripts\build.bat exe` (compiles via PyInstaller & Inno Setup).
- **Build Android APK:** Run `npm run build:apk` in `apps/mobile/app/` or use EAS Build.

13. Diagnostics & Troubleshooting

Observed Issue	Root Cause	Recommended Resolution
Mobile app cannot reach PC	Windows Defender Firewall blocking ports 8000/8080/8090	Run <code>scripts/start.bat</code> once as Administrator to register inbound firewall rules.
QR Code shows 172.x / 127.0.0.1	Virtual NIC (Docker / WSL / Hyper-V) prioritised	Disable unused virtual adapters in Windows Network Connections, or set <code>BACKEND_HOST</code> .
Screen stream is black	Frame buffer truncation or display DPI scaling mismatch	Update graphics drivers; adjust <code>STREAM_MAX_WIDTH</code> & <code>STREAM_MAX_HEIGHT</code> in <code>.env</code> .
Port in use conflict	Stale python process holding port	Kill lingering processes with <code>taskkill /F /IM python.exe</code> .

14. Test Suite & Quality Verification

The repository includes a pytest unit test suite covering domain entities, pairing logic, and mock input drivers:

```
# Execute all unit tests
pytest tests/unit/

# Target specific modules
pytest tests/unit/core/test_pairing.py
pytest tests/unit/core/test_control_input.py
pytest tests/unit/core/test_controller.py
```